PRACTICAL – 4

---

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

   Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code: (4A)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    // Create first child
    pid1 = fork();

    if (pid1 == 0) {
        printf("Child 1\n");
        printf("PID  : %d\n", getpid());
        sleep(3);
        printf("Child 1 Finished\n");
        exit(0);
    }

    // Create second child
    pid2 = fork();

    if (pid2 == 0) {
        printf("Child 2\n");
        printf("PID  : %d\n", getpid());
        sleep(5);
        printf("Child 2 Finished\n");
        exit(0);
    }

    printf("\nParent PID : %d\n", getpid());

    // Wait for any child
    wait(NULL);
```

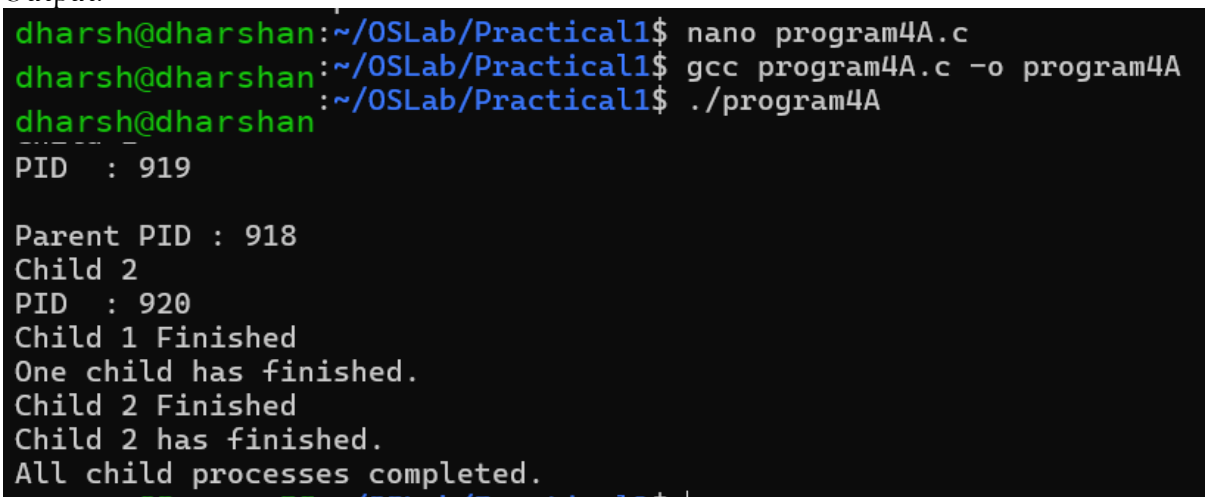
```
    printf("One child has finished.\n");

    // Wait for the second child
    waitpid(pid2, NULL, 0);
    printf("Child 2 has finished.\n");

    printf("All child processes completed.\n");

    return 0;
}
```

Output:



PART_B :- Zombie Process

Code: (4B):

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {

    pid_t pid = fork();
```

```
    if (pid == 0) {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());

        printf("Child exiting...\n");
        exit(0);
    }

    else {
        printf("Parent PID : %d\n", getpid());

        printf("Parent sleeping...\n");

        sleep(20);

        printf("Parent finished.\n");
    }

    return 0;
}
```

Output: